Changelog: v1 → v2 Rebuild

This document describes what changed between the original codebase (archived in `archive/`) and the current version, why each change was made, and what issues it corrects.

Summary

The v1 codebase had the right modular structure but was optimizing against **placeholder and surrogate cost functions that did not reflect real VBAP physics**. The VBAP implementation used dot-product similarities instead of triangulation-based panning. The objective function's VBAP term returned a constant `1.0`. The optimizer was a 3000-iteration simulated annealing loop — severely underpowered for a 72-dimensional search space. Only a single listener position was evaluated.

The v2 rebuild replaces every stage of the pipeline:

- **Real VBAP** via ConvexHull triangulation with 8 perceptual quality metrics
- **Multi-listener evaluation** across an 18-position 3D grid with weighted-worst aggregation
- **Differential Evolution** with multi-restart, stagnation detection, and L-BFGS-B polish (~1M function evaluations vs. 3000)
- **Adaptive cost weighting** that focuses optimization pressure where improvement is most needed
- **Variable speaker counts** with automated sweep and comparison
- **Comprehensive analysis notebook** with 12 visualization/analysis sections and automated report generation

---

Issue 1: VBAP Was Not Actually VBAP

Problem (v1)

`archive/spatial_audio/vbap.py` contained two functions — `compute_vbap_amplitudes_ideal()` and `compute_vbap_amplitudes()` — that were **identical**. Both computed raw dot products between the desired direction and each speaker direction:

```
amplitude = np.dot(speaker_direction, test_direction_unit)
```

The cost was the squared error between these two identical outputs, meaning **J_vbap was always approximately 0** and provided no optimization signal.

Real VBAP requires:

1. A **Delaunay triangulation** of speaker directions into speaker triplets
2. For each desired direction, finding the **enclosing triangle**
3. Solving a **3×3 linear system** for the panning gains
4. Only **2–3 speakers active** per direction (sparse activation)

The v1 surrogate missed all of these properties and could not detect coverage gaps, degenerate triangles, or localization errors.

Fix (v2)

`src/spatial_audio/vbap.py` now contains a `VBAPRenderer` class that implements real 3D VBAP:

- Builds a `scipy.spatial.ConvexHull` on speaker unit directions — the hull facets **are** the VBAP speaker triplets
- Precomputes L^{-1} (the inverse of each triangle's 3×3 direction matrix) at construction time
- For each test direction, finds the enclosing triangle by checking $g = L^{-1} \cdot d \geq 0$
- Returns proper VBAP gains with energy normalization ($g / \sqrt{\sum(g^2)}$)
- Computes **rV** (velocity vector) and **rE** (energy vector) for perceptual localization assessment

The `evaluate_vbap_quality()` function returns 8 quality metrics, all normalized to [0, 1]:

Metric	What it measures
Coverage	Fraction of test directions with a valid enclosing triangle
Localization error	Mean rV angular error (low-frequency direction accuracy)
Energy error	Mean rE angular error (high-frequency direction accuracy)
rV magnitude error	Deviation of |rV| from 1.0 (localization sharpness)
rE magnitude error	Deviation of |rE| from 1.0 (energy sharpness)
Conditioning	Mean condition number of active triangles (numerical stability)
Angular uniformity	Variance of triangle solid angles (evenness)
Max gap	Largest angular gap between hull-adjacent speakers

The evaluation is **vectorized** using `numpy.einsum` for batch gain computation across all test directions, providing significant speedup over per-direction Python loops.

Directions below the horizon are **down-weighted** by $\cos(\text{elevation})^2$ to prevent the floor gap (where no speakers can physically be placed) from dominating the aggregate metrics.

Issue 2: The Active Cost Function Was a Placeholder

Problem (v1)

`archive/optimization/objective.py` contained `score_vbap_for_listener()` which **returned a constant `1.0`** with a comment: "*Replace this with your real VBAP metric later.*" The separate `vbap.py` implementation was never called by the cost function. `score_regularization()` also returned a constant `1.0`.

The cost function structure was:

```
J_total = (w_vbap * J_vbap + w_hoa * J_hoa + w_off * J_off + w_reg * J_reg) /
sum(weights)
```

With weights `vbap=1, hoa=0, off=0, reg=0`, the optimizer was minimizing:

```
J_total = (1.0 * 1.0) / 1.0 = 1.0  (constant)
```

No optimization was happening. The only active signal came from the HOA surrogate (when its weight was nonzero), which used `max(0, cos(θ))^4 / distance` as a gain model — not real Ambisonics decoding.

Fix (v2)

`src/optimization/objective.py` now calls `evaluate_vbap_quality()` from the real VBAP module. The cost function:

- Evaluates all 8 VBAP metrics at each of the 18 listener positions
- Combines them with configurable weights (coverage=5.0 is highest priority, followed by loc_error=5.0 and max_gap=2.0)
- Aggregates across listeners via **weighted worst-case**: `0.7 × mean + 0.3 × worst`
- Adds a mild **distance penalty** for speakers far from the listening area
- Adds a **coverage floor penalty** — steep cost when any listener position drops below 95% coverage
- Returns a detailed breakdown of all metrics for diagnostics

The HOA surrogate module (`hoa_cost_terms.py`) is retained in the codebase but **removed from the active cost pipeline**, with a prominent docstring marking it as legacy. The rV/rE analysis now operates on real VBAP gains.

Issue 3: Simulated Annealing Was Underpowered for 72D

Problem (v1)

The optimizer in `archive/optimization/search.py` ran **3,000 iterations** of simulated annealing for a **72-dimensional** search space (24 speakers × 3 coordinates). This is severely undersampled — typical SA convergence for this dimensionality requires orders of magnitude more evaluations.

Additional issues:

- All 24 speakers were **perturbed simultaneously** per iteration, making acceptance unlikely (most speakers get worse even when some improve)
- `T_init = 0.05` with costs in [0, 1] gave very low initial acceptance probability (~37% for a +0.05 cost increase)
- Linear cooling to `T = 0.0` made the search fully greedy by the end
- No **multi-restart strategy** to escape local optima
- No **gradient information** exploited (the cost function is differentiable w.r.t. positions)
- The perturbation used a **jitter-with-retry** strategy that could waste many iterations retrying infeasible moves, then "teleporting" a speaker to a random location

Fix (v2)

`src/optimization/search.py` provides **four optimizer backends**, all using scipy or CMA-ES:

1. `optimize_layout_de()` — Multi-restart Differential Evolution
    - Population-based: maintains ~1,080 candidate solutions simultaneously (for 24 speakers with popsize=15)
    - ~1M function evaluations per restart (vs. 3,000 total in v1)
    - `randtobest1bin` strategy balancing exploration and exploitation
    - Dithered mutation `(0.5, 1.5)` for adaptive step sizing
    - L-BFGS-B polish after DE convergence for local refinement
    - Stagnation detection: terminates a restart early if no improvement for 20 generations
    - 3 independent restarts with different seeds; best result kept
2. `optimize_layout_multistart()` — Multi-start L-BFGS-B
    - Evaluates 20-100 random feasible layouts in Phase 1
    - Refines top 5 with L-BFGS-B gradient-based optimization in Phase 2
    - Much faster than DE for problems where the landscape is relatively smooth
3. `optimize_layout_da()` — Dual Annealing
    - Combines generalized simulated annealing for global exploration with L-BFGS-B for local refinement
    - Proper scipy implementation (vs. the hand-rolled SA in v1)
4. `optimize_layout_cmaes()` — CMA-ES (Covariance Matrix Adaptation)
    - Population-based global optimizer that adapts its search distribution
    - Better than DE for problems with correlated variables
    - Requires the optional `cma` package

All optimizers use a **repair+penalty hybrid** for constraint handling:

- Each infeasible speaker is projected to the nearest valid box via `repair_to_feasible()`
- A penalty proportional to total repair distance is added to the cost
- Minimum speaker distance violations use a separate soft penalty
- An angular spread penalty (Tammes-like) provides gradient signal even when VBAP metrics plateau
- A symmetry penalty discourages extreme left-right speaker count imbalance

All optimizers support **adaptive weight scaling**: before optimization, random layouts are sampled and evaluated. Metrics far from their realistic ideal receive higher weight; metrics already near ideal are downweighted. Realistic ideals account for room geometry (e.g., ~27° localization error is the floor imposed by the missing floor speakers).

Issue 4: Single Listener Position

Problem (v1)

Only one listener at `[0, 0, 1.2]` was used. The off-center term (`J_off`) existed in the cost function but was disabled (`off=0`). In a CAVE where users walk around, optimizing for a single point produces a layout that may perform poorly at other positions — VBAP triangulation changes as the listener moves because speakers subtend different angles from different vantage points.

Fix (v2)

A **3×3×2 listener grid** is generated across 60% of the CAVE floor area at two heights (0.9m seated, 1.5m standing), giving **18 evaluation positions**. The cost function evaluates VBAP quality at every grid position and aggregates via weighted worst-case:

```
cost = 0.7 × mean(per_listener_costs) + 0.3 × max(per_listener_costs)
```

This balances average performance with robustness at the worst position. A **coverage floor penalty** adds steep cost when any listener drops below 95% coverage.

Issue 5: Fixed Speaker Count

Problem (v1)

The number of speakers was hardcoded at 24. There was no way to explore the quality-vs-count tradeoff or determine if fewer speakers could achieve acceptable performance.

Fix (v2)

The orchestration script (`src/recipes/test_run.py`) runs optimization for each count in `SPEAKER_COUNTS = [16, 18, 20, 22, 24]`. Each count is an independent optimization. Results are saved as JSON and compared via plots of cost, coverage, localization error, and max gap vs. speaker count. This directly reveals the diminishing returns curve and supports informed decisions about speaker count.

Issue 6: Constraint Handling for Population-Based Optimizers

Problem (v1)

SA used feasibility-preserving perturbations — if a jittered position was infeasible, it was rejected and retried (up to `max_retries=20`). If retries failed, jitter was increased; if that also failed, the speaker was "teleported" to a random valid location. This worked but was wasteful: many iterations were spent retrying infeasible moves, and teleportation destroyed any local structure the optimizer had built.

Fix (v2)

A **repair+penalty hybrid** in `src/optimization/search.py`:

1. **Repair**: Each infeasible speaker is projected to the nearest valid box boundary via `repair_to_feasible()` (clamp to box bounds, check forbidden regions, pick closest). This ensures every candidate evaluated by the cost function is physically meaningful.

2. **Feasibility penalty**: Proportional to total repair distance (`sum of ||original - repaired||`), normalized by speaker count and room scale. This creates a smooth gradient that guides the optimizer toward naturally feasible solutions.

3. **Distance penalty**: `sum(max(0, min_dist - d_ij))` for all speaker pairs closer than the minimum separation. Soft penalty allows the optimizer to transiently violate the constraint while being pushed away.

4. **Angular spread penalty**: A Tammes-like metric that penalizes poor angular distribution of speakers as seen from the listener. Measures the minimum nearest-neighbor angle and variance of nearest-neighbor angles on the unit sphere. This provides gradient signal even when VBAP metrics plateau.

5. **Symmetry penalty**: `|n_right - n_left| / n_total` discourages extreme left-right speaker count imbalance.

`src/optimization/speaker_placement.py` was extended with:

- `repair_to_feasible()` — nearest-feasible-point projection
- `generate_feasible_initial_population()` — generates a full DE population of valid layouts
- Maximum placement attempt limit (`50,000`) with clear error messaging

---

Issue 7: The HOA Implementation Was a Surrogate

Problem (v1)

`archive/spatial_audio/hoa_cost_terms.py` used a surrogate gain model:

```
g_i = max(0, dot(speaker_i, desired))^sharpness / distance^power
```

While physically motivated, this is **not real Ambisonics decoding** — there is no spherical harmonic decomposition, no decode matrix, and no order-dependent behavior. The rV/rE analysis was applied to these surrogate gains rather than to actual VBAP or Ambisonics gains, making the localization metrics unreliable.

Fix (v2)

The HOA module is retained in the codebase with a prominent **"LEGACY — retained for reference only"** docstring and a warning not to import it in production code. The rV/rE analysis now operates on **real VBAP gains** from the ConvexHull triangulation, making the localization metrics physically meaningful.

If real Ambisonics support is needed in the future, it should be implemented with proper spherical harmonic encoding/decoding at a specified order — the surrogate cannot substitute for this.

Issue 8: No Visualization or Analysis Pipeline

Problem (v1)

`archive/utils/plot_layout.py` existed but only showed a static 3D scatter of the final layout. There was no way to visualize the triangulation, coverage gaps, localization errors, convergence, or the VBAP sound field. The

`plot_cost_terms_over_time()` function in `archive/optimization/search.py` referenced a `cost_terms_over_time` variable that was never populated (the tracking list `layouts` and `scores` were defined but never appended to during the optimization loop).

Fix (v2)

`speaker_optimization_viz.ipynb` provides a comprehensive 12-section interactive analysis notebook:

Section	Content
1. Setup & Configuration	Editable parameters, import paths
2. Room Geometry & Listener Grid	3D room with allowed/forbidden regions, listener positions
3. Initial Layout & VBAP Triangulation	Random starting layout with its triangulation on the unit sphere
4. Coverage Heatmap	Per-direction coverage and localization error sphere
5. Optimization	Run inline (DE, multistart, CMA-ES) or load from CLI results
6. Convergence Plots	4-panel: total cost, VBAP cost, repair cost, distance violations
7. Before vs After Layout	Side-by-side 3D comparison with metrics table
8. Optimized Triangulation & Coverage	Unit sphere heatmap of the final layout
9. Sound Field Rendering	For representative source directions: gain distribution, desired vs perceived direction arrows
9a. Frequency-Dependent Perception	rV vs rE comparison spheres, per-elevation bars, room-view arrows for 6 representative directions
10. Metrics Summary	Detailed metric tables and deep-dive analyses:
10a. Coverage Gap Investigation	Per-listener coverage, worst-case gap classification by hemisphere
10b. Left-Right Asymmetry Analysis	Feasible region geometry explaining the speaker distribution
10c. Reference Layout Comparison	Optimized vs ITU 7.1.4 and extended 9.1.6 standards
10d. Per-Elevation Breakdown	Quality metrics in 30° elevation bands (reveals floor gap impact)
10e. Floor Speaker Analysis	Hypothetical floor speakers quantifying the ceiling of improvement
11. Speaker Count Sweep	Multi-count comparison with non-monotonic cost diagnosis
12. Report Export	Automated generation of `SPEAKER_PLACEMENT_REPORT.md`

Both **static matplotlib** and **interactive Plotly** versions are provided for key visualizations. The notebook supports two modes: run optimization inline, or load results from a previous CLI run.

A new utility module `src/utils/results_summary.py` handles JSON results saving and comparison plotting across speaker counts.

Issue 9: No Adaptive Weighting

Problem (v1)

Cost weights were static: `vbap=1, hoa=0, off=0, reg=0`. Even if all four terms had been active, a static weighting cannot account for the fact that some metrics are already near their ideal while others are far away. The optimizer would spend effort improving already-good metrics at the expense of poor ones.

Fix (v2)

`src/optimization/objective.py` contains `compute_adaptive_weights()` which:

1. Evaluates 5-10 random layouts to measure baseline metric values
2. Computes the gap between each metric and its **realistic ideal** (not the theoretical ideal — e.g., ~27° localization error is the realistic floor for a room with no floor speakers)
3. Scales each weight by `gap / mean_gap`, clamped to [0.3, 3.0]
4. Enforces a **coverage weight floor** (never below 60% of base) — coverage is a hard requirement
5. Coordinates the `spread_penalty` and `max_gap` weights to avoid double-counting

---

Files Changed

File	Change	What happened
`src/spatial_audio/vbap.py`	**Rewritten**	Dot-product surrogate → real ConvexHull VBAP with 8 metrics, vectorized evaluation
`src/optimization/objective.py`	**Rewritten**	Placeholder (`return 1.0`) → multi-listener VBAP cost function with adaptive weights and coverage floor
`src/optimization/search.py`	**Rewritten**	3,000-iter hand-rolled SA → 4 optimizer backends (DE, multistart L-BFGS-B, dual annealing, CMA-ES) with repair+penalty constraint handling
`src/optimization/speaker_placement.py`	**Extended**	Added `repair_to_feasible()`, `generate_feasible_initial_population()`, max attempt limits
`src/config/config.py`	**Rewritten**	SA params → DE params, 8 VBAP cost weights + adaptive flag, listener grid config, speaker count list, stagnation/penalty settings

File	Change	What happened
`src/recipes/test_run.py`	**Rewritten**	Single fixed-count run → outer loop over speaker counts, listener grid generation, JSON results saving
`src/spatial_audio/hoa_cost_terms.py`	**Marked legacy**	Added "LEGACY — do not import" docstring; module retained for reference but removed from active pipeline
`src/utils/results_summary.py`	**New**	JSON results saving + multi-count comparison plotting
`speaker_optimization_viz.ipynb`	**New**	12-section interactive analysis notebook with static and Plotly visualizations
`SPEAKER_PLACEMENT_REPORT.md`	**New**	Comprehensive report generated from notebook analyses

Unchanged: `src/geometry/box.py`, `src/geometry/define_spaces.py`, `src/geometry/geometry_utils.py`, `src/utils/plot_layout.py`, `run.py`

Archived: All original v1 source files preserved in `archive/` with original directory structure